electronics

MDPI

Article

Span-Prototype Graph Based on Graph Attention Network for Nested Named Entity Recognition

Jichong Mu [1], Jihong Ouyang [2,*], Yachen Yao [3] and Zongxiao Ren [4]

[1] College of Software, Jilin University, Changchun 130022, China; mujc21@mails.jlu.edu.cn
[2] College of Computer Science and Technology, Jilin University, Changchun 130022, China
[3] School of Software, Dalian University of Technology, Dalian 116600, China
[4] College of Petroleum Engineering, Xi'an Shiyou University, Xi'an 710065, China
* Correspondence: oujh@jlu.edu.cn

Abstract: Named entity recognition, a fundamental task in natural language processing, faces challenges related to the sequence labeling framework widely used when dealing with nested entities. The span-based method transforms nested named entity recognition into span classification tasks, which makes it an efficient way to deal with overlapping entities. However, too much overlap among spans may confuse the model, leading to inaccurate classification performance. Moreover, the entity mentioned in the training dataset contains rich information about entities, which are not fully utilized. So, in this paper, a span-prototype graph is constructed to improve span representation and increase its distinction. In detail, we utilize the entity mentions in the training dataset to create a prototype for each entity category and add prototype loss to adapt the span to its similar prototype. Then, we feed prototypes and span into a graph attention network (GAT), enabling span to automatically learn from different prototypes, which integrate the information about entities into the span representation. Experiments on three common nested named entity recognition datasets, including ACE2004, ACE2005, and GENIA, show that the proposed method achieves 87.28%, 85.97%, and 79.74% F1 scores on ACE2004, ACE2005, and GENIA, respectively, performing better than baselines.

Keywords: nested named entity recognition; span classification; prototype; graph attention network

Citation: Mu, J.; Ouyang, J.; Yao, Y.; Ren, Z. Span-Prototype Graph Based on Graph Attention Network for Nested Named Entity Recognition. *Electronics* **2023**, *12*, 4753. https://doi.org/10.3390/electronics12234753

Academic Editor: Fernando De la Prieta Pintado

Received: 11 September 2023
Revised: 12 October 2023
Accepted: 25 October 2023
Published: 23 November 2023

1. Introduction

Named entity recognition (NER) is an essential task in natural language processing, which aims to detect and classify spans of text into pre-defined entity categories, such as location (LOC), person (PER), geo-political (GPE), organization (ORG), and so on [1]. Span means a sequence of text of varying lengths, consisting of consecutive tokens in a sentence, so it could be a single token or more tokens. The extracted entities can be used for downstream tasks such as machine translation, question and answer systems, information retrieval, text classification, and keyword ranking [2], as well as for more advanced language processing and analysis. Previous studies usually view NER as a sequential labeling problem, which strictly stipulates that each token belongs to one entity mention at most. Thus, it is just suitable for flat NER not nested NER. It is obvious that there are great limitations because entity overlapping is quite common in text [3]. Taking Figure 1 as an example, the entity "Homeland Security" with label "ORG" is nested in the entity "Secretary of Homeland Security" with label "PER". Both of them are further nested in the entity "Secretary of Homeland Security Tom Ridge" of type "PER". Hence, better methods are required to handle this complexity. Research on nested named entity recognition is helpful to provide richer semantic representation and capture the nested structure between entities, so as to better understand the entity hierarchy in text and improve the richness of semantic representation.

Figure 1. An example of nested NER in the ACE2005 dataset.

Various research into nested NER has been proposed in recent years, which can be primarily divided into three categories: hypergraph-based approaches, layered-based approaches, and span-based approaches. (a) Hypergraph-based approaches [4–6] rely on the idea that an arbitrary number of vertices can be connected with hyperedges to model the possible nested structure. But it will be pretty complex when encountering structural ambiguity. (b) Layered-based approaches [7–9] solve nested NER problems by stacking multiple flat NER layers to transform nested NER into flat NER. However, this model suffers from serious error propagation. If the internal entity in the previous layer cannot be correctly identified, then the external entity in the next layer will not be detected; (c) span-based approaches [10–12] take the most direct way to classify text spans enumerated from the sentence, which are widely used in recent years.

Although span-based methods have achieved good performance in nested NER tasks, they also face some serious defects. Firstly, they encounter challenges such as deterioration in scenarios where positive instances and negative instances substantially overlap [12]. The model may have difficulty recognizing the spans with slight difference. Secondly, since a large portion of the test set entities are rarely present in the training set, the significance of the generalization ability is emphasized in nested NER [13]. Thirdly, span-based methods usually only focus on span content, ignoring available information beyond the current sentence.

In this paper, a span-prototype graph is proposed to alleviate the problems mentioned above, which aims at improving the representation of span and increasing the differentiation of easily confused spans. Different from other span-based methods, the methods proposed in this work not only focus on the internal content of span but also concatenate the boundary information and fully utilize the entity mention information in the training set. In particular, we do not use dependency trees or introduce any external knowledge, improving the utilization of the existing data. To be specific, firstly, all the extracted spans are initially screened, and those with low correlation with entities are eliminated to prevent too many negative samples from interfering with the model classification. Secondly, in the training dataset, create the prototypes by averaging the entity mentions' representations by the same class to represent each category. Then, the prototype loss is added to make span adapt to the prototype representation, which is similar to it. Thirdly, treat span and prototypes as nodes, then adopt graph attention network (GAT) to jointly learn the representations of span and prototypes, making the span learn from prototypes that are highly relevant to entities and improving classification performance. Following the aforementioned operations, the span representation will be endowed with more effective information, obtaining an enhanced classification efficacy.

In summary, the main work of this paper is as follows:

From all possible spans, we filter out those that have little correlation with entities and reserve the more potentially valuable candidates that are relevant to the entities.

We create the prototype in the total training dataset, which is utilized to represent the representation of a certain class of entities. In the process, not only is no complex training process required, but it also does not depend on any specific generative model. Then, we leverage prototype loss to teach the model to learn the semantic representations required for spans.

We construct a span-prototype graph and feed the nodes into a graph attention network (GAT), so that the span is able to extract rich information about entities carried by the prototypes and find a better representation through the iterative propagation of messages between the nodes of the GAT, improving the classification performance of the model.

Experiments on three common nested NER datasets, namely ACE2004, ACE 2005, and GENIA, show that the proposed model outperforms baselines. In addition, the ablation experiments demonstrate the effectiveness of each module.

The remaining sections of this paper are organized as follows. Section 2 introduces the related works of named entity recognition, nested named entity recognition, and graph attention network. Section 3 describes the problem definition and the proposed model in detail. Section 4 lists experimental settings and comprehensively analyses the experimental results. Section 5 provides the conclusion and future outlooks of this work.

2. Related Work

2.1. NER

Named entity recognition (NER) is frequently used as a preprocessing step for natural language processing tasks, providing important information for other downstream tasks. Before deep learning becomes popular, early named entity recognition mainly includes two methods: rule-based and statistics-based methods. Among the rule-based methods, Shaalan et al. [14] use the context features of the text to construct rules and add a toponymy dictionary to identify professional terms. Krupka et al. [15] design a SRA system for English NER. In statistics-based methods, they use HMM (hidden Markov models), CRF (conditional random fields), MEM (maximum entropy models), and so on. Bikel et al. [16] propose a statistical learning method with a variant of the standard hidden Markov model to find names and other non-recursive entities in text. McCallum et al. [17] propose a NER method of feature induction based on CRF. Borthwick et al. [18] apply MEM to NER by comprehensively considering a variety of characteristic information, such as sentence ending information and initial letters. In recent years, the rise of deep learning has injected new vitality into NER, which helps to automatically discover hidden features. Collobert et al. [19] utilize CNN to encode tokens and CRF to classify them. Lample et al. [20] obtain the representation of the whole sentence via BiLSTM and then label each word through the CRF layer. Zhang et al. [21] propose a Lattice-LSTM model to embed lexical information into each character through a gating unit. Ma et al. [22] improve the Lattice-LSTM model, only integrating words with all matched lexical information at the input layer without modifying the internal structure of LSTM. However, all of them are applied to flat NER but not to nested NER.

2.2. Nested NER

Nested named entity recognition has captured the attention of numerous researchers in recent years. In this section, we will briefly introduce three existing main methods to tackle the task related to our work: hypergraph-based, layered-based, and span-based approaches.

Hypergraph-based: The nested named entity recognition task is modeled as a path search problem in a hypergraph. Lu et al. [4] propose the idea that a hypergraph enables edges to establish connections with multiple nodes, providing a way to describe a nested structure. Muis et al. [23] introduce a mention hypergraph for nested NER, further developing a gap-based tagging schema, which assigns labels to spaces between words. Wang et al. [5] attempt to utilize a neural segmental hypergraph model to eliminate structural ambiguity. Luo et al. [24] suggest a method to grasp reciprocal information exchanges among layers in a hypergraph structure. However, hypergraph-based models face the defects of complex and ambiguous structures.

Layered-based: Ju et al. [7] dynamically stack flat NER layers to update representations for subsequent layer recognition based on the identification of internal entities. Fisher et al. [25] merge the representation from low-level token embedding to high-level entity embedding. Shibuya et al. [9] expand on the previous second-best path recognition approach by specifically disregarding the impact of the best path. Wang et al. [8] propose a pyramid structure, which consists of forward and reverse pyramids, to recursively put the text region into flat NER layers. Yang et al. [1] improve the representation of the text region by fusing context information based on a bottom-up and top-down transformer network

with a two-phase module. But, as for the layered-based methods, if an error occurs in the lower layer, it will propagate to the next layer.

Span-based: It utilizes the straight and simple way to transform the nested NER problem into span classification, which has become popular in recent years. Sohrab et al. [26] enumerate all spans from the text and classify them with a deep neural network. Li et al. [27] connect NER to machine reading comprehension, providing the answers for given questions. Tan et al. [28] propose a sequence-to-set model, which no longer gives the span in advance but provides a fixed set of learnable vectors to learn span. Shen et al. [10] regard the nested NER problem as an object detection task, locating entities and predicting the types. Xu et al. [29] consider the correlation strength of paired words in sentences under each entity type and apply a supervised multi-head self-attention mechanism to predict the span type. Huang et al. [30] introduce a span selection framework to extract different classes of nested entities, and a discriminator is designed to evaluate the extraction results. Although span-based methods in overlapping tasks have achieved nice performance, they also face the phenomenon of a large overlap among spans that will confuse the model, reducing classification performance.

2.3. Graph Attention Network

Graph attention network (GAT) [31] is a deep learning model for processing graph data, which learns the relationships between nodes by introducing attention mechanisms and dynamically assigns weights during information transferring, thus more effectively capturing complex relationships between nodes. The GAT can use multiple attention heads to capture relationships at different levels. Each attention head models the relationship between the nodes differently and then enriches the representation of the nodes by merging the output from multiple heads [32]. The core idea of the GAT is to introduce different weights for connections between each pair of nodes in order to adaptively adjust the importance of nodes during information transferring. In other words, the GAT is more flexible than traditional graph convolutional networks (GCN) [33].

3. Model

In this part, the proposed model will be introduced in detail. First of all, we show the formulation of nested named entity recognition (NNER) task as follows:

Problem formulation of NNER. Given a sentence $X_s = \left\{x_1, x_2, \cdots, x_{|N|}\right\}$, where x_i is a token denoting to a word in X_s, and $|N|$ is the length of the sentence. We extract all spans into a span set $S = \{x_{lk} | 1 \leq l \leq k \leq |N|\}$ and entity mentions in the training set into an entity set $E = \{e_{se} | 1 \leq s \leq e \leq |N|\}$, where x_{lk} indicates a span from x_l to x_k, and e_{se} means an entity from x_s to x_e. Let Y_t denote the list of all entity categories. Following Eberts et al. [34], we formulate the NNER problem as the span classification task to classify x_{lk} into the pre-defined entity types y_l,$y_l \in Y_t$.

3.1. Overall Framework

Figure 2 presents the overview of the proposed model structure. Overall, it consists of four components. (1) Encoder: in this part, we concatenate word embedding, character embedding, context embedding, and part-of-speech embedding to obtain the final representations of spans and entity mentions via a BiLSTM. (2) Filter: the filter module is utilized to drop low-quality spans, which may bring some disturbing noise to the model. (3) Prototype Module: after obtaining the representations from the encoder, we create the prototypes by averaging the representations of entity mentions with the same class. And then, we utilize GAT to build a bridge between span and prototypes for better span representation. (4) Classifier: it classifies the spans into pre-defined entity types.

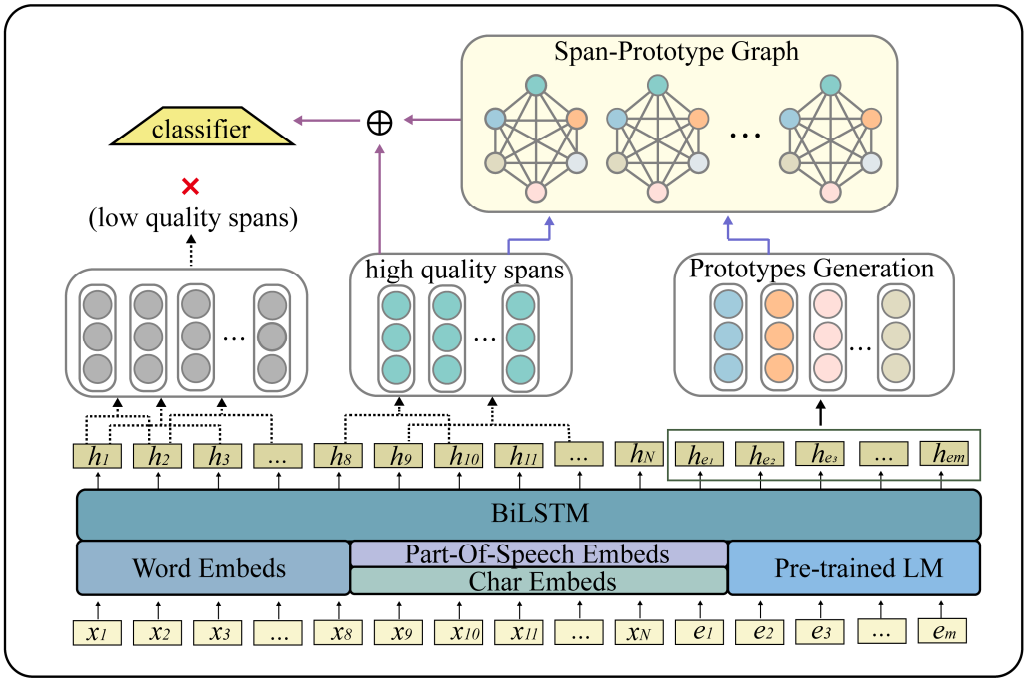

Figure 2. The overall architecture of the model.

3.2. Encoder Module

In this module, sentence and entity are treated as a sequence of tokens. For each word, we first find the word embedding $\widehat{h_{word}}$ through GloVe or BioWordvec. Then, a BiLSTM is used to obtain the character embedding $\widehat{h_{char}}$. Considering that the contextual information is informative, we follow Yu at al. [35] to obtain the context embedding $\widehat{h_{context}} = \mathcal{F}(X_s)$ for a target token, with surrounding tokens each side, through BERT or BioBERT, where $\mathcal{F}(\cdot)$ denotes the pre-trained-LM [36] (e.g., BERT (Devlin et al. [37])). Finally, after concatenating $\widehat{h_{word}}$, $\widehat{h_{char}}$, $\widehat{h_{context}}$ and the part-of-speech embedding $\widehat{h_{pos}}$ annotated via the Stanford Core NLP [38], another BiLSTM receives concatenation and generates the final representation of each word $h_i \in \mathbb{R}^D$, in which D is the hidden demension. h_i denotes the hidden state of each word and can be calculated by

$$h_i = \text{BiLSTM}\left(\widehat{h_{word}} \oplus \widehat{h_{char}} \oplus \widehat{h_{context}} \oplus \widehat{h_{pos}}\right) \tag{1}$$

where "$\oplus$" is the concatenating operation.

Specially, the representations of spans and entities are generated via the same encoder but encoded separately. For the span, since boundary information is useful for entity classification, we concatenate max-pooling span representation $\widehat{H_{lk}}$ with the boundary representation $h_l \in \mathbb{R}^D$ and $h_k \in \mathbb{R}^D$. Thus, we get the final span representation H_{lk} by

$$\widehat{H_{lk}} = MaxPooling(h_l, h_{l+1}, h_{l+2} \ldots, h_k) \tag{2}$$

$$H_{lk} = \widehat{H_{lk}} \oplus h_l \oplus h_k \tag{3}$$

In the same way, we obtain the final entity representation E_{se} by

$$\widehat{E_{se}} = MaxPooling(h_s, h_{s+1}, h_{s+2} \ldots, h_e) \tag{4}$$

$$E_{se} = \widehat{E_{se}} \oplus h_s \oplus h_e \tag{5}$$

3.3. Filter

As we enumerate all possible spans, a large proportion of the spans has low correlation with entities. If all the spans participate in classification, they will inevitably bring a serious imbalance of positive and negative samples, and too much noise may interfere with the classification performance of the model. Thus, we divide all spans into two categories: spans that are more relevant to the entity and spans that are less relevant to the entity. The latter spans will be dropped. Moreover, inspired by [39], focal loss is added to alleviate the problem of positive and negative sample imbalance.

$$p_i^f = Sigmoid\Big(g_1\Big(\widehat{H_{lk}}W_1 + b_1\Big)\Big) \tag{6}$$

$$\mathcal{L}_f = -\sum_i 1\big[\widehat{y} \neq 0\big]\alpha(1-p_i^f)^\gamma log p_i^f + 1\big[\widehat{y} = 0\big](1-\alpha)\big(p_f{}^\gamma\big)log(1-p_i^f) \tag{7}$$

In Equations (6) and (7), g_1 is the GELU activation function, W_1 and b_1 are the trainable parameters, i is the ith span, 1 is indicator function, $\widehat{y}$ is the ground-truth, α is the hyperparameter that controls the imbalance in the number of positive and negative samples, and γ is the hyperparameter that can adjust the weight relationship between easy and difficult samples.

3.4. Prototype Module

3.4.1. Prototype Learning

The basic idea of creating prototypes is to represent each class as a prototype vector, giving a uniform representation of a category. We calculate the prototype $\xi_t \in \mathbb{R}^D$ for each class by averaging the feature representations of the entity mentions, which share the same label t in Equations (8) and (9). In particular, the prototypes are created in the entire training dataset. Here, a prototype is defined as a representative embedding for a group of semantically similar instances.

$$\xi_t = \frac{1}{|\mathcal{D}_t|}\sum_{(X_s,E,Y_t)}\sum_{y_l \in Y_t} 1\big[y_l = t\big]E_{se} \tag{8}$$

$$|\mathcal{D}_t| = \sum_{(X_s,E,Y_t)}\sum_{y_l \in Y_t} 1\big[y_l = t]E_{se} \tag{9}$$

where $|\mathcal{D}_t|$ is the cardinality of entities with the same class t,E_{se} is the representation of the entity, and 1[·] is the indicator function. Next, for the span representation $\widehat{H_{lk}}$, the prototype loss $\mathcal{L}_p$ is computed by

$$\mathcal{L}_p = -\sum_{H_{lk} \in S} log p\big(y_l|H_{lk}\big) \tag{10}$$

$$p\big(y_l|H_{lk}\big) = softMax(-d(H_{lk}|\xi_t)) \tag{11}$$

In Equation (11), $p\big(y_l|H_{lk}\big)$ is the probabilistic distribution, and $d(\cdot,\cdot)$ is the Euclidean distance function.

3.4.2. Span-Prototype Graph

The graph attention network (GAT) utilizes a multi-head attention mechanism to aggregate information from neighbors, which achieves the information delivery. In addition, the attention mechanism assigns more weight to those with more relevance. In order to make full use of prototypes' information and obtain more effective span representation, the span-prototype graph with the GAT is designed. In detail, we treat the span H_{lk} and the prototypes ξ_t obtained in the previous section as the nodes of the graph $X = (\xi_1, \xi_2, \ldots, \xi_m, H_{lk})$. Then, we construct a fully connected adjacency matrix, namely $M \in \mathbb{R}^{(m+1)\times(m+1)}$ and $M_{i,j} = M_{j,i} = 1$.

In the GAT, the node embedding vector x_i^{l+1} at the layer $l+1$ is calculated by x_j^l at layer l as follows:

$$x_i^{l+1} = \sum_{j \in \mathcal{N}(i)} \alpha_{i,j} W_x x_j^l \tag{12}$$

$$\alpha_{i,j} = \frac{exp\left(e_{i,j}\right)}{\sum_{j \in \mathcal{N}(i)} exp\left(e_{i,j}\right)} \tag{13}$$

where $\mathcal{N}(i)$ is the set of one-hop neighbors of node i, and W_x is a learnable matrix. $\alpha_{i,j}$ is the attention score between the nodes i and j, which can be calculated using Equation (14):

$$e_{i,j} = LeakyReLu(W_a(W_f x_i \oplus W_f x_j)) \tag{14}$$

$$\overrightarrow{x_i} = \sigma(\frac{1}{K} \sum_{k=1}^{K} \sum_{j \in \mathcal{N}(i)} \alpha_{i,j}^k W^k x_j) \tag{15}$$

In Equation (15), σ is the activation function; K is the number of multi-heads. $\oplus$ is the concatenation; W_f, W_a, and W^k are the learnable matrix; and $\overrightarrow{x_i}$ is the final representation of x_i.

Finally, we can obtain the graph representation $\overrightarrow{X} = GAT(X, M)$ for span H_{lk}.

3.5. Classifier

For span H_{lk}, we concatenate the hidden vector H_{lk} and the graph representation $\overrightarrow{X}$ to obtain the output representation $\mathcal{H}_{lk}$ towards H_{lk}:

$$\mathcal{H}_{lk} = H_{lk} \oplus \overrightarrow{X} \tag{16}$$

Then, we put it into the classifier. For this multi-class task, we compute the probability distribution for each category via $softMax$:

$$p_i = softMax\left(g_2(\mathcal{H}_{lk} W_2 + b_2)\right) \tag{17}$$

where g_2 is GELU activation function, and W_2 and b_2 are the trainable parameters. We adopt a cross-entropy loss between predicted distribution p_i and ground-truth distribution $\widehat{y}$ of the span to train the classifier.

$$\mathcal{L}_{class} = -\sum_{i=1}^{T} \widehat{y} log\left(p_i\right) \tag{18}$$

where T is the total number of classes.

3.6. Training Objective

The model is optimized by minimizing the loss function during training. In this work, the training objective of the model contains three losses: $\mathcal{L}_p$, $\mathcal{L}_{class}$, and $\mathcal{L}_f$. Thus, the overall loss $\mathcal{L}$ can be formulated, as Equation (19) shows.

$$\mathcal{L} = \beta_1 \mathcal{L}_{fl} + \beta_2 \mathcal{L}_p + \beta_3 \mathcal{L}_{class} \tag{19}$$

where β_1, β_2, and β_3 are the coefficient parameters.

4. Experiment and Analysis

In this section, a series of experiments are conducted on three nested English NER datasets, including ACE2004, ACE2005, and GENIA, to illustrate the validity of the proposed model. We will introduce the datasets, parameter settings, evaluation, baselines and main results, ablation experiments, and sensitivity analysis of our proposed model.

4.1. Datasets

We conduct experiments on ACE2004, ACE2005, and GENIA. The ACE2004 and ACE2005 are various types of datasets made up of entity, relationship, and event annotations published by the Language Data Consortium (LDC), all of which include seven types of entities: PER, ORG, GPE, LOC, FAC, VEH, and WEA. The GENIA is a biological nested named entity recognition dataset, including five types of entities: DNA, RNA, protein, cell line, and cell type. In addition, the nesting ratio of GENIA is about 17%, while the nesting ratio of ACE2004 and ACE2005 is about 35~45%.

We follow the train/validation/test split of a previous works by Yu et al. [35]: (1) ACE 2004 and ACE 2005—we split the data into 80%, 10%, and 10% as the training, development, and test sets, respectively. (2) GENIA: We use 90% and 10% for the training and testing split. More dataset statistics in the above three datasets are shown in Table 1. We note that 'sent. nested entities' means the number of sentences with nested entities, and 'avg. sentence length' means the average length of the sentences in the dataset.

Table 1. Statistics of ACE 2005, ACE 2004, GENIA in the experiments.

	ACE2005			**ACE2004**			**GENIA**	
	Train	**Dev**	**Test**	**Train**	**Dev**	**Test**	**Train**	**Test**
total sentences	7194	969	1047	6200	745	812	16,692	1854
total entities	24,441	3200	2993	22,204	2514	3035	50,509	5506
sent. nested entities	2691	338	320	2712	294	388	3522	446
avg. sentence length	19.21	18.93	17.2	22.50	23.02	23.05	25.35	25.99
total nested entities	9389	1112	1118	10,149	1092	1417	9064	1199
nested percentage (%)	38.41	34.75	37.35	45.71	46.69	45.61	17.95	21.78

4.2. Evaluation

The NER task involves two basic steps, which are detecting entity boundaries and determining entity classes. The entity is considered correct if both span and class are predicted right. We adopt micro-precision (P), micro-recall (R) and micro-F1 scores (F1) for evaluation, which are calculated by

$$\mathrm{P} = \frac{\widetilde{TP}}{\widetilde{TP} + \widetilde{FP}} \tag{20}$$

$$\mathrm{R} = \frac{\widetilde{TP}}{\widetilde{TP} + \widetilde{FN}} \tag{21}$$

$$\mathrm{F1} = \frac{2 * \mathrm{P} * \mathrm{R}}{\mathrm{P} + \mathrm{R}} \tag{22}$$

$\widetilde{TP}$ represents the number of samples that are actually positive and predicted to be positive, $\widetilde{FP}$ represents the number of samples that are actually negative and predicted to be positive, and $\widetilde{FN}$ represents the number of samples that are actually positive but predicted to be negative.

4.3. Parameter Settings

We use BERT for contextual embedding in ACE 2004 and ACE 2005 and BioBERT in GENIA. The hidden size of BERT/BioBERT is 1024 dimensional. For the word embedding, we use 100-dimensional GloVe embeddings trained for ACE2004 and ACE2005 and 100-dimensional BioWordvec embeddings trained for GENIA. The BiLSTM we utilize is 1024 dimensional. Moreover, we use 50-dimensional char embeddings and 50-dimensional part-of-speech embeddings. We choose the Adam optimizer with a learning rate of 3×10^{-5}, and the weight decay is set to 0.01 in ACE2004 and ACE2005. In addition, for GENIA, we use a 5×10^{-6} learning rate. The length of the span that we enumerate is 1~10 in GENIA and 1~15 in ACE, which can cover almost all entity lengths. As for the graph attention network,

the dimension of the feature is 1024, the number of attention heads is 4, and the dropout rate is 0.1. α and γ are set to 0.25 and 2.0, and β_1, β_2, β_3 are set to 1.0, 0.3, 1.0. For these three datasets, we set the batch size to 10, 8, 6 for ACE2004, ACE2005, GENIA, respectively, and train the model for 40 epochs.

4.4. Baselines and Results

4.4.1. Baseline Methods

In order to evaluate the effect of the model proposed in this paper, the model is compared to several mainstream and representative baseline models. The comparison models involved are briefly introduced as follows:

HyperGraph [40]: uses features extracted from recurrent neural networks to learn hypergraph representations of nested entities;

Second-Path [9]: treats the sequence of labels of nested entities as the second-best path within the span of their parent entities and iteratively extracts entities from the outermost layer to the interior;

Pyramid [8]: proposes a novel layered structure to extract nested entity recognition, and token region embeddings are recursively input into the flat NER layers;

BENSC [12]: adds the boundary detection task to predict the words of the entity boundary, which effectively enhances the span representation;

TreeCRFs [41]: treats the nested entity structure as a partially observed selection tree and models it with partially observed TreeCRFs;

Point Network [42]: proposes a unified pointer network-based approach to solve both component parsing and nested named entity recognition, which encodes both component syntax trees and nested named entities;

BartNER [43]: expresses NER subtasks as an entity generation problem across sequences and utilizes a unified Seq2Seq model and pointer mechanism to deal with flat, nested, and discontinuous NER subtasks.

4.4.2. Experimental Result

Table 2 shows the main results on the ACE 2004, ACE 2005, and GENIA compared to the baselines, and the best results are shown in bold.

Table 2. Results for nested NER tasks.

Models	ACE 2004			ACE 2005			GENIA		
	P	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
HyperGraph (2018)	73.60	71.80	72.70	70.60	70.40	70.50	77.70	71.80	74.60
Second-path (2020)	85.94	85.69	85.82	83.83	84.87	84.34	77.81	76.94	77.36
Pyramid (2020)	86.08	86.48	86.28	83.95	85.39	84.66	79.45	78.94	79.19
BENSC (2020)	85.80	84.80	85.30	83.80	83.90	83.90	79.20	77.40	78.30
TreeCRFs (2021)	86.70	86.50	86.60	84.50	86.40	85.40	78.20	78.20	78.20
BartNER (2021)	**87.27**	86.41	86.84	83.16	86.38	84.74	78.87	79.60	79.23
PointNetwork (2022)	86.60	87.28	86.94	84.61	**86.43**	85.53	78.08	78.26	78.16
Ours	87.17	**87.40**	**87.28**	**85.71**	86.23	**85.97**	**79.48**	**80.01**	**79.74**

In order to more intuitively show the comparative experimental results, we use the bar chart to describe the experimental results in Figures 3–5.

From the above graphs and Table 2, it is obvious that our model performs generally better than those baseline models. Here, we mainly analyze P and F1, where P measures the number of instances that the model predicts correctly and F1 reflets the harmonic average of P and R. On ACE 2004, the HyperGraph model performs the worst, which may have a lot to do with its structural ambiguity. Meanwhile, BartNER finds the best in P, and our model is the second best. But in F1, which is a more important metric than P, our model achieves the best results, better balancing P and R. Although the ACE 2004 dataset has a high nesting rate, our model also surpasses the relatively better PointNetwork and

BartNER by 0.34% and 0.44% in F1, respectively. On ACE 2005, our model is slightly lower than PointNetwork in R but performs the best in terms of P and R, surpassing others in P by at least 1% and in F1 at least 0.43%. On GENIA dataset, our model continues to perform best, achieving 79.48%, 80.01%, and 79.74% in P, R, and F1. The experimental results are the mean values of 10 experiments. And in the statistical results of these 10 experiments, the proposed model has a significant improvement compared to the baseline methods. Overall, our model achieves competitive results on the three datasets compared to the baseline approaches. Through the analysis of the experimental results, it is enough to prove that the proposed model in this paper can obtain more efficient span representation by integrating prototype information, which represents similar semantics for the same class. Meanwhile, it also shows that the full use of entity mentions information in training set can effectively improve the classification performance of the model.

4.5. Ablation Study

In order to verify the improvement in the performance of the proposed model via different modules, we set ablation experiments for three groups to verify the effectiveness of the boundary, filter, and prototype, respectively, where w/o means "not included". As shown in Table 3, from top to bottom are erasing boundary representation, erasing filter module, erasing prototype module, and full model.

Table 3. Ablation results on ACE 2004, ACE 2005, and GENIA.

Model	ACE 2004			ACE 2005			GENIA		
	P	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
w/o boundary	86.88	87.21	87.04	85.36	86.20	85.78	79.12	79.80	79.46
w/o filter	86.53	87.30	86.91	84.97	85.85	85.41	78.83	80.20	79.51
w/o prototype	86.14	87.18	86.66	84.32	85.91	85.11	78.16	79.65	78.90
Full model	87.17	87.40	87.28	85.71	86.23	85.97	79.48	80.01	79.74

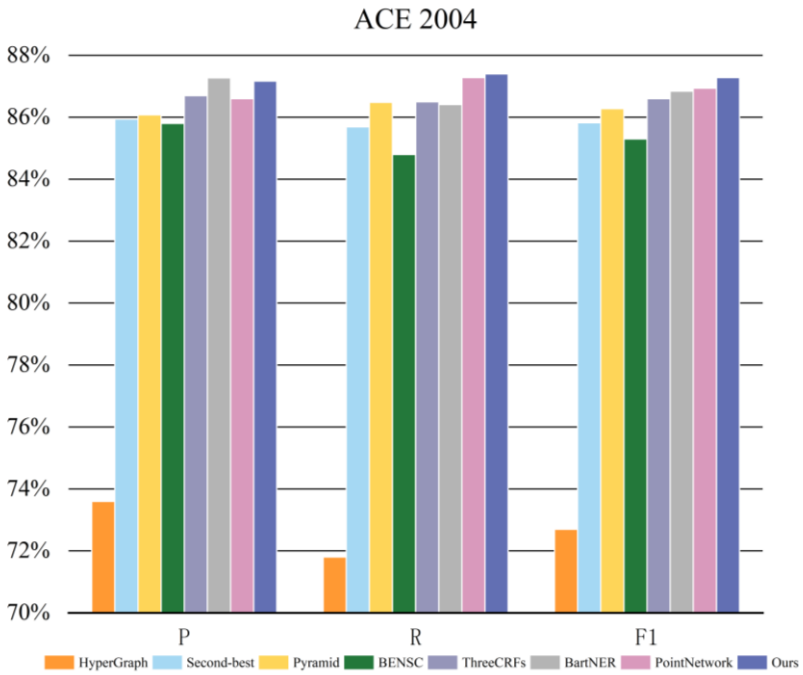

Figure 3. The experimental results of the ACE 2004 dataset.

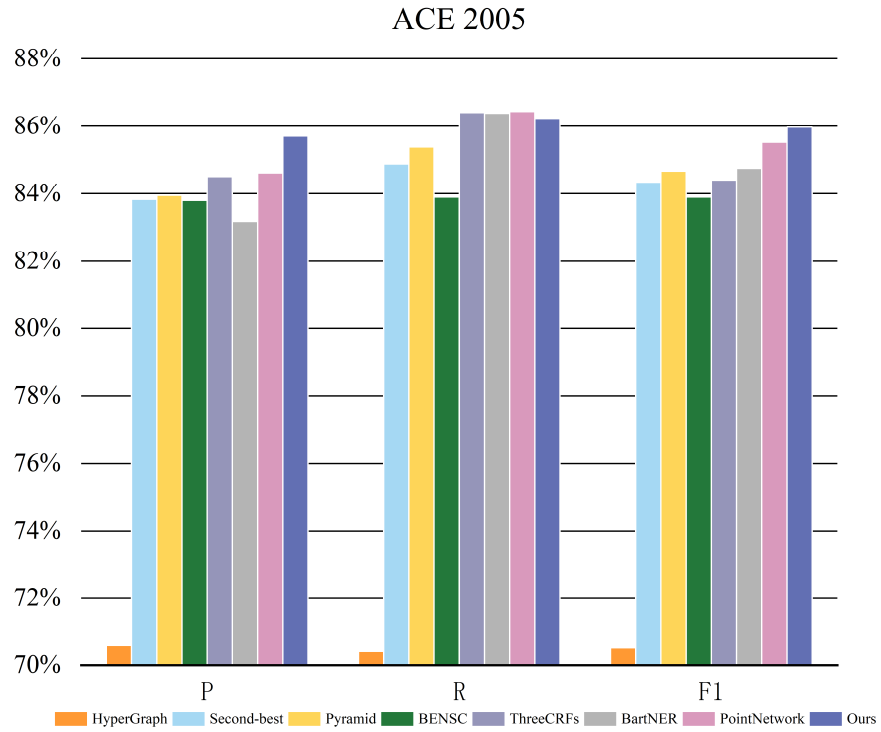

Figure 4. The experimental results of the ACE 2005 dataset.

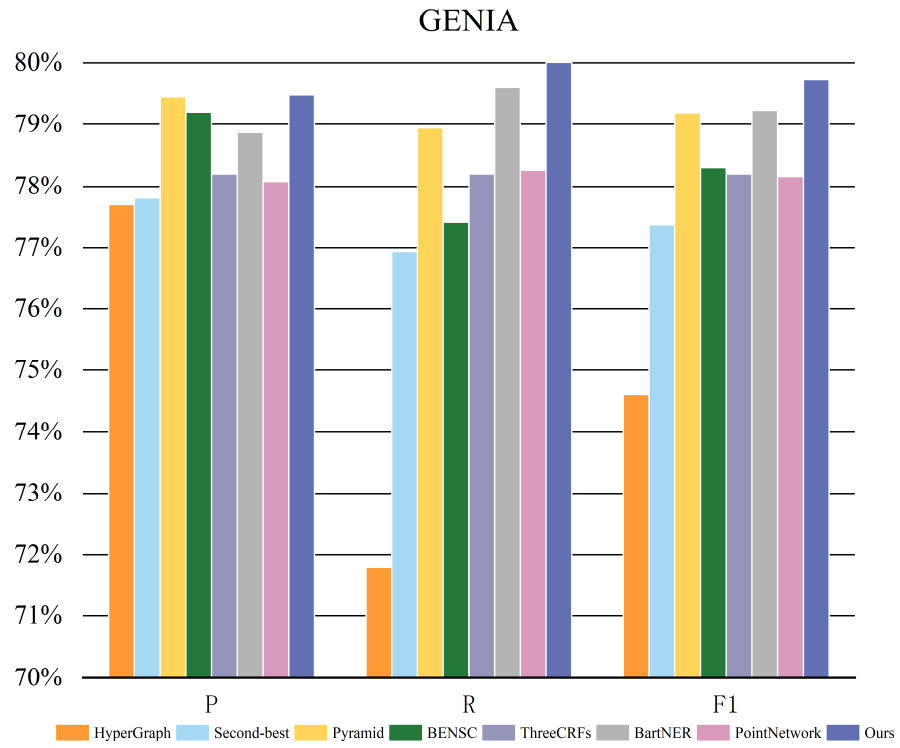

Figure 5. The experimental results of the GENIA dataset.

Also, for more clarity, we draw bar graphs, as shown in Figures 6–8 for different datasets.

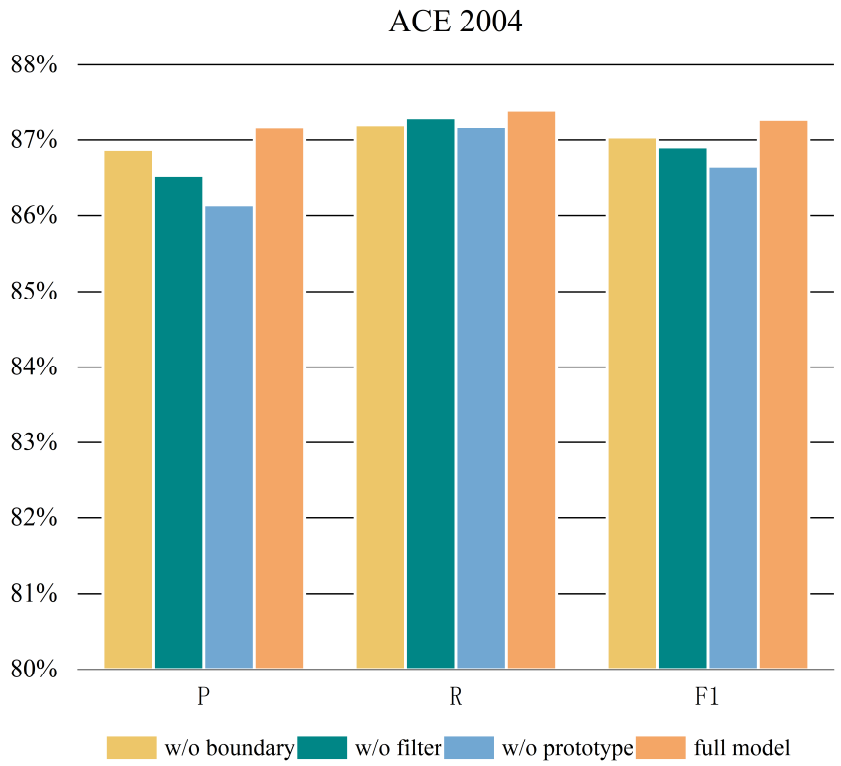

Figure 6. The ablation results of the ACE 2004 dataset.

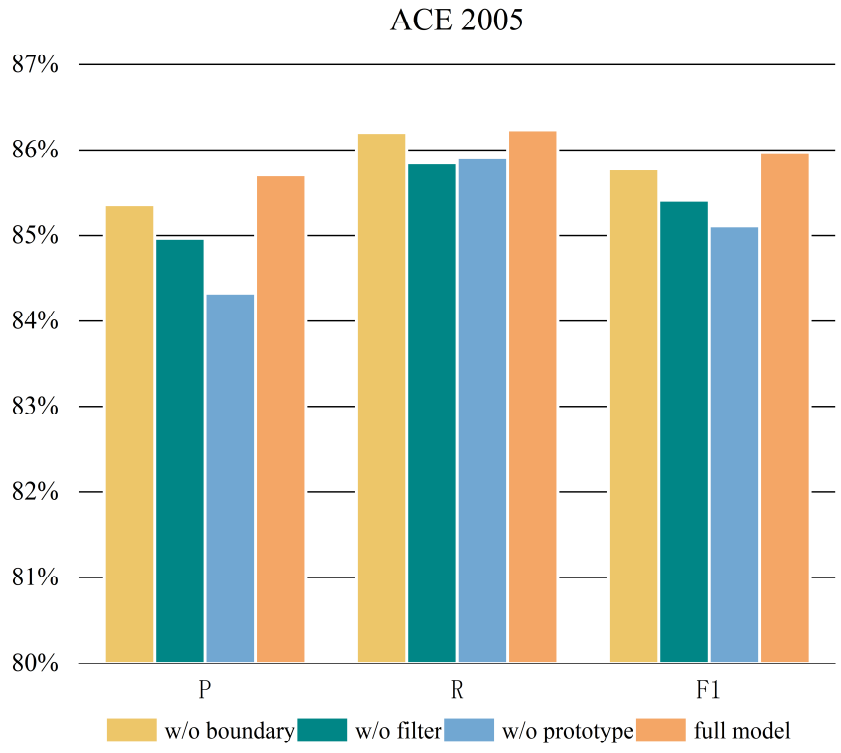

Figure 7. The ablation results of the ACE 2005 dataset.

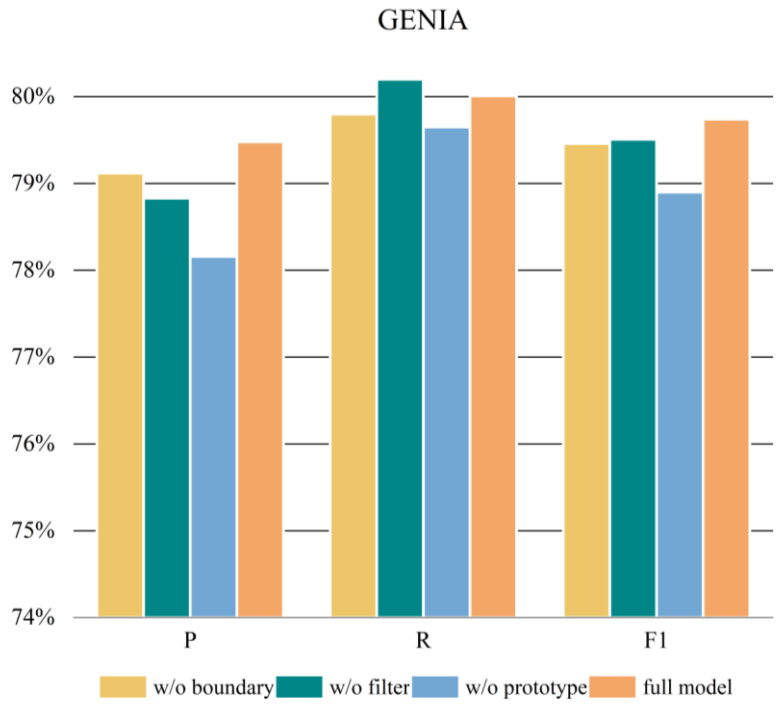

Figure 8. The ablation results of the GENIA dataset.

As demonstrated in Figures 6–8 and Table 3, when some modules are removed, the performance of the model decreases to varying degrees. As we can see, the removal of the prototype module is the most obvious, dropping 1.03%, 1.29%, and 1.32% in P and 0.62%, 0.86%, and 0.84% in F1 on ACE 2004, ACE 2005, and GENIA, respectively. Filter module is the second, dropping 0.64%, 1.14%, and 0.65% in P and 0.37%, 0.38%, and 0.23% in F1 on ACE 2004, ACE 2005, and GENIA, respectively. Although the effect of removing the boundary is not as significant as the other two, it also decreases by 0.24%, 0.21%, and 0.28% in F1 on ACE 2004, ACE 2005, and GENIA. The experimental results not only demonstrate the effectiveness of each module but also prove that the prototype is the most effective module, which makes span fully learn entity information from the prototype and improves the ability of model classification.

4.6. Sensitivity Analysis

4.6.1. The Layers of Graph Attention Network (GAT)

In this section, we will a conduct sensitivity analysis of the layer of the graph attention network (GAT) to explore the influence on F1. In the experiment, the number of layers is set from 1 to 5 on ACE 2004, ACE 2005, and GENIA. Figure 9 shows the F1 at different number of layers.

Just as illustrated in Figure 9, the best result is obtained when the number of layers is 2, while the worst result is obtained when the number of layers is 4 on GENIA and 5 on ACE. In addition, with the further increase in layers, the effect decreases, possibly due to the disappearance of the gradient or excessive smoothing.

4.6.2. The Multi-Heads of GAT

In this section, we will conduct sensitivity analysis of the multi-heads of the GAT to explore the influence on F1. In the experiment, the number of multi-heads is set to 1, 2, 4, 8, and 16 on ACE 2004, ACE 2005, and GENIA. Figure 10 shows the F1 at different numbers of multi-heads.

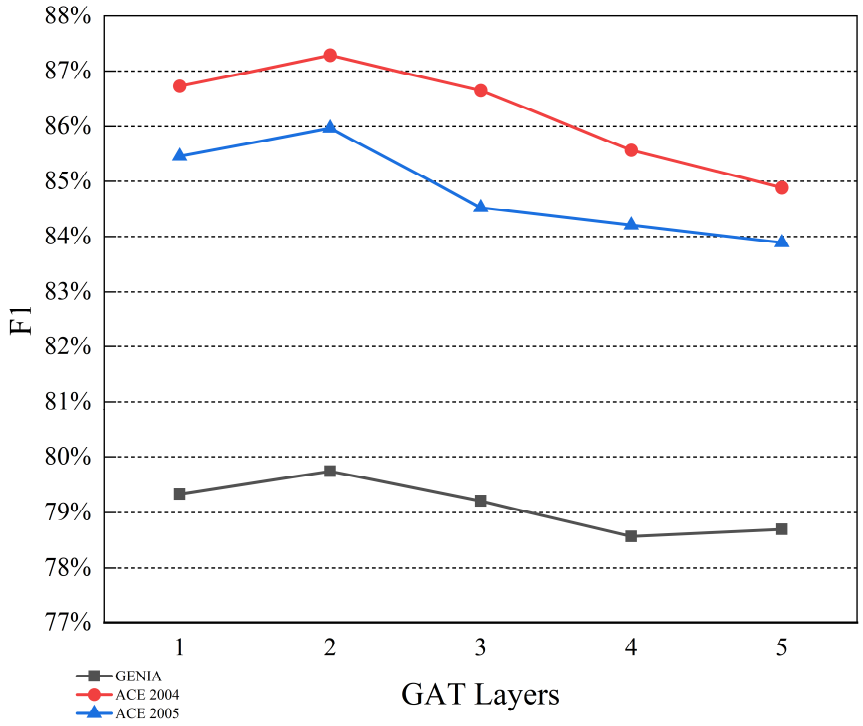

Figure 9. Sensitivity analysis of GAT layers.

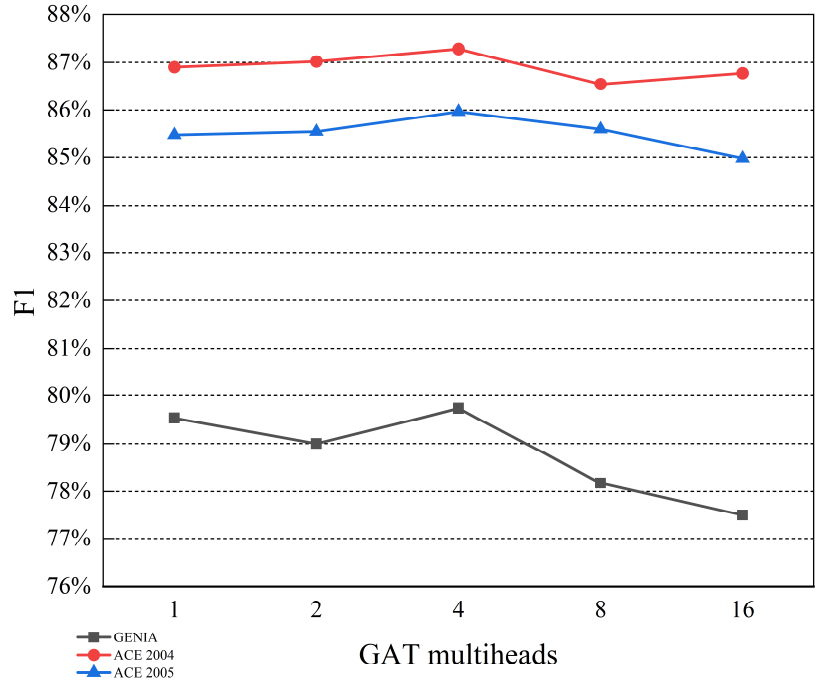

Figure 10. Sensitivity analysis of GAT multi-heads.

From Figure 10, it is obvious that the best result is achieved when the number of heads is 4, and too many or too few attention heads both lead to a decrease in results. That is because too many attention heads may make the model inclined to overfit, while the model may not adequately capture diversity in the input data with too few heads.

4.6.3. Comparison of Different Span Representation Enhancement Methods

We compare the method of improving span representation in this paper with another method. In detail, instead of the GAT, we calculate the distance between a span and each

prototype, where the distance adopted is Euclidean distance. Then, the prototype closest to the span is concatenated with the span representation, and, thus, the final representation of the span is obtained. For convenience, this method without the GAT is denoted as M1, and the method with the GAT in this paper denoted as M2. Figure 11 shows the comparison of the two methods on F1.

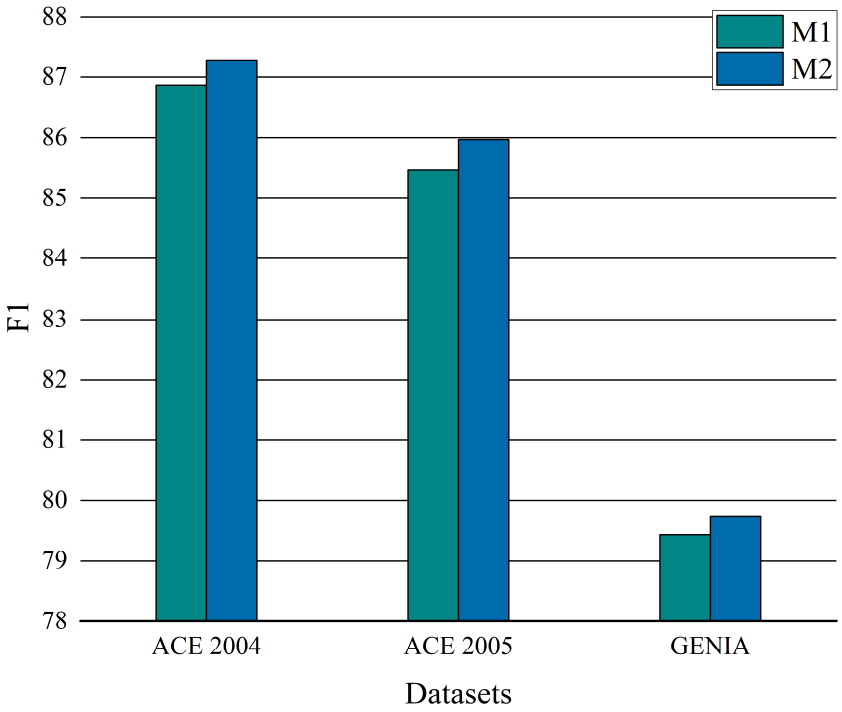

Figure 11. The results of M1 and M2 on F1.

As can be seen from Figure 11, the method proposed in this paper achieves higher F1, which indicates that the GAT enables the span to selectively learn prototype representation to improve its representation and the classification performance of the model.

5. Conclusions

In this paper, the span-based method is adopted for tackling the complex task of nested named entity recognition. To elevate the quality of span representations and improve the classification ability of the model, we introduce a novel element known as the span-prototype graph. The model creates prototypes for entities to denote semantically similar entity class representations and then construct a span-prototype graph as a bridge between span and prototypes. In particular, the graph attention network plays a vital role in facilitating information propagation across the span-prototype graph. Through this mechanism, each span embedding within the graph can autonomously learn distinct prototype feature representations, thereby subsequently improving its differentiation from the others. Experimental results on ACE2004, ACE2005, and GENIA demonstrate the efficacy of the proposed method, as evidenced by achieving F1 scores of 87.28%, 85.97%, and 79.74% on the respective datasets. Furthermore, the results of comparative experiments, as well as ablation studies, consistently affirm the effect and significance of the model proposed in this work for nested named entity recognition. Moreover, the usage of computing resources is a point that needs to be further optimized. In the future, we will explore the application of a graph attention network for discontinuous entity named recognition.

Author Contributions: J.M.: conceptualization, methodology, experiments, original draft; J.O.: conceptualization, methodology, supervision, review, funding; Y.Y.: software, original draft, data processing; Z.R.: review. All authors have read and agreed to the published version of the manuscript.

Funding: The work is supported by the National Natural Science Foundation of China (NSFC) (No.61876071, No.62006094) and Scientific and Technological Developing Scheme of Jilin Province (No.20180201003SF, No.20190701031GH) and Energy Administration of Jilin Province (No.3D516L921421).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Yang, Z.; Ma, J.; Chen, H.; Zhang, Y.; Chang, Y. HiTRANS: A Hierarchical Transformer Network for Nested Named Entity Recognition. In *Findings of the Association for Computational Linguistics: EMNLP 2021*; Association for Computational Linguistics: Punta Cana, Dominican Republic, 2021; pp. 124–132. [CrossRef]
2. Chen, L.-C.; Chang, K.-H. An Extended AHP-Based Corpus Assessment Approach for Handling Keyword Ranking of NLP: An Example of COVID-19 Corpus Data. *Axioms* **2023**, *12*, 740. [CrossRef]
3. Finkel, J.R.; Manning, C.D. Nested Named Entity Recognition. In Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing Volume 1—EMNLP '09, Singapore, 2–7 August 2009; Volume 1, p. 141. [CrossRef]
4. Lu, W.; Roth, D. Joint Mention Extraction and Classification with Mention Hypergraphs. In Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, Lisbon, Portugal, 17–21 September 2015; Association for Computational Linguistics: Lisbon, Portugal, 2015; pp. 857–867. [CrossRef]
5. Wang, B.; Lu, W. Neural Segmental Hypergraphs for Overlapping Mention Recognition. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 Octobe–4 November 2018; Association for Computational Linguistics: Brussels, Belgium, 2018; pp. 204–214. [CrossRef]
6. Straková, J.; Straka, M.; Hajič, J. Neural Architectures for Nested NER through Linearization. *arXiv* **2019**, arXiv:1908.06926.
7. Ju, M.; Miwa, M.; Ananiadou, S. A Neural Layered Model for Nested Named Entity Recognition. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), New Orleans, LA, USA, 1–6 June 2018; Association for Computational Linguistics: New Orleans, LA, USA, 2018; pp. 1446–1459. [CrossRef]
8. Wang, J.; Shou, L.; Chen, K.; Chen, G. Pyramid: A Layered Model for Nested Named Entity Recognition. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 5918–5928. [CrossRef]
9. Shibuya, T.; Hovy, E. Nested Named Entity Recognition via Second-Best Sequence Learning and Decoding. *Trans. Assoc. Comput. Linguist.* **2020**, *8*, 605–620. [CrossRef]
10. Shen, Y.; Ma, X.; Tan, Z.; Zhang, S.; Wang, W.; Lu, W. Locate and Label: A Two-Stage Identifier for Nested Named Entity Recognition. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Online, 1–6 August 2021; pp. 2782–2794. [CrossRef]
11. Zhong, Z.; Chen, D. A Frustratingly Easy Approach for Entity and Relation Extraction. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 6–11 June 2021; pp. 50–61. [CrossRef]
12. Tan, C.; Qiu, W.; Chen, M.; Wang, R.; Huang, F. Boundary Enhanced Neural Span Classification for Nested Named Entity Recognition. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 9016–9023. [CrossRef]
13. Wan, J.; Ru, D.; Zhang, W.; Yu, Y. Nested Named Entity Recognition with Span-Level Graphs. In Proceedings of the Annual Meeting of the Association for Computational Linguistics, Dublin, Ireland, 22–27 May 2022; Volume 1, pp. 892–903.
14. Shaalan, K.; Raza, H. NERA: Named Entity Recognition for Arabic. *J. Am. Soc. Inf. Sci.* **2009**, *60*, 1652–1663. [CrossRef]
15. Krupka, G.R. SRA: Description of the SRA System as Used for MUC-6. In Proceedings of the 6th Conference on Message understanding—MUC6 '95, Columbia, MA, USA, 6–8 November 1995; Association for Computational Linguistics: Columbia, MA, USA, 1995; p. 221. [CrossRef]
16. Bikel, D.M.; Miller, S.; Schwartz, R.; Weischedel, R. Nymble: A High-Performance Learning Name-Finder. In Proceedings of the Fifth Conference on Applied Natural Language Processing, Washington, DC, USA, 31 March–3 April 1997; Association for Computational Linguistics: Washington, DC, USA, 1997; pp. 194–201. [CrossRef]
17. McCallum, A.; Li, W. Early Results for Named Entity Recognition with Conditional Random Fields, Feature Induction and Web-Enhanced Lexicons. In Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, Edmonton, AB, Canada, 31 May–1 June 2003; Association for Computational Linguistics: Edmonton, AB, Canada, 2003; Volume 4, pp. 188–191. [CrossRef]
18. Borthwick, A.; Sterling, J.; Agichtein, E.; Grishman, R. NYU: Description of the MENE Named Entity System as Used in MUC-7. In Proceedings of the 7th Message Understanding Conference, MUC 1998—Proceedings, Fairfax, VA, USA, 29 April–1 May 1998.
19. Collobert, R.; Weston, J.; Bottou, L.; Karlen, M.; Kavukcuoglu, K.; Kuksa, P. Natural Language Processing (Almost) from Scratch. *J. Mach. Learn. Res.* **2011**, *12*, 2493–2537.
20. Lample, G.; Ballesteros, M.; Subramanian, S.; Kawakami, K.; Dyer, C. Neural Architectures for Named Entity Recognition. *arXiv* **2016**, arXiv:1603.01360.

21. Zhang, Y.; Yang, J. Chinese NER Using Lattice LSTM. *arXiv* **2018**, arXiv:1805.02023.
22. Ma, R.; Peng, M.; Zhang, Q.; Wei, Z.; Huang, X. Simplify the Usage of Lexicon in Chinese NER. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–20 July 2020; pp. 5951–5960. [CrossRef]
23. Muis, A.O.; Lu, W. Labeling Gaps Between Words: Recognizing Overlapping Mentions with Mention Separators. In Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, Copenhagen, Denmark, 7–11 September 2017; pp. 2608–2618. [CrossRef]
24. Luo, Y.; Zhao, H. Bipartite Flat-Graph Network for Nested Named Entity Recognition. *arXiv* **2020**, arXiv:2005.00436.
25. Fisher, J.; Vlachos, A. Merge and Label: A Novel Neural Network Architecture for Nested NER. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; Association for Computational Linguistics: Florence, Italy, 2019; pp. 5840–5850. [CrossRef]
26. Sohrab, M.G.; Miwa, M. Deep Exhaustive Model for Nested Named Entity Recognition. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018; Association for Computational Linguistics: Brussels, Belgium, 2018; pp. 2843–2849. [CrossRef]
27. Li, X.; Feng, J.; Meng, Y.; Han, Q.; Wu, F.; Li, J. A Unified MRC Framework for Named Entity Recognition. *arXiv* **2022**, arXiv:1910.11476.
28. Tan, Z.; Shen, Y.; Zhang, S.; Lu, W.; Zhuang, Y. A Sequence-to-Set Network for Nested Named Entity Recognition. In Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, Montreal, QC, Canada, 19–26 August 2021; International Joint Conferences on Artificial Intelligence Organization: Montreal, QC, Canada, 2021; pp. 3936–3942. [CrossRef]
29. Xu, Y.; Huang, H.; Feng, C.; Hu, Y. A Supervised Multi-Head Self-Attention Network for Nested Named Entity Recognition. In Proceedings of the AAAI Conference on Artificial Intelligence AAAI 2021, Online, 2–9 February 2021; Volume 35, pp. 14185–14193. [CrossRef]
30. Huang, P.; Zhao, X.; Hu, M.; Fang, Y.; Li, X.; Xiao, W. Extract-Select: A Span Selection Framework for Nested Named Entity Recognition with Generative Adversarial Training. In *Findings of the Association for Computational Linguistics: ACL 2022*; Association for Computational Linguistics: Dublin, Ireland, 2022; pp. 85–96. [CrossRef]
31. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; Bengio, Y. Graph Attention Networks. *arXiv* **2018**, arXiv:1710.10903.
32. Liang, S.; Wei, W.; Mao, X.-L.; Wang, F.; He, Z. BiSyn-GAT+: Bi-Syntax Aware Graph Attention Network for Aspect-Based Sentiment Analysis. In *Findings of the Association for Computational Linguistics: ACL 2022*; Association for Computational Linguistics: Dublin, Ireland, 2022; pp. 1835–1848. [CrossRef]
33. Kipf, T.N.; Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv* **2017**, arXiv:1609.02907.
34. Eberts, M.; Ulges, A. Span-Based Joint Entity and Relation Extraction with Transformer Pre-Training. In Proceedings of the 24th European Conference on Artificial Intelligence (ECAI), Santiago de Compostela, Spain, 29 August–8 September 2020; Volume 325, pp. 2006–2013.
35. Yu, J.; Bohnet, B.; Poesio, M. Named Entity Recognition as Dependency Parsing. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 6470–6476. [CrossRef]
36. Lybarger, K.; Yetisgen, M.; Uzuner, Ö. The 2022 N2c2/UW Shared Task on Extracting Social Determinants of Health. *J. Am. Med. Inform. Assoc.* **2023**, *30*, 1367–1378. [CrossRef]
37. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2019**, arXiv:1810.04805.
38. Manning, C.; Surdeanu, M.; Bauer, J.; Finkel, J.; Bethard, S.; McClosky, D. The Stanford CoreNLP Natural Language Processing Toolkit. In Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Baltimore, MA, USA, 23–24 June 2014; Association for Computational Linguistics: Baltimore, MA, USA, 2014; pp. 55–60. [CrossRef]
39. Zheng, Q.; Wu, Y.; Wang, G.; Chen, Y.; Wu, W.; Zhang, Z.; Shi, B.; Dong, B. Exploring Interactive and Contrastive Relations for Nested Named Entity Recognition. *IEEE/ACM Trans. Audio Speech Lang. Process.* **2023**, *31*, 2899–2909. [CrossRef]
40. Katiyar, A.; Cardie, C. Nested Named Entity Recognition Revisited. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), New Orleans, LA, USA, 1–6 June 2018; Association for Computational Linguistics: New Orleans, LA, USA, 2018; pp. 861–871. [CrossRef]
41. Fu, Y.; Tan, C.; Chen, M.; Huang, S.; Huang, F. Nested Named Entity Recognition with Partially-Observed TreeCRFs. *arXiv* **2020**, arXiv:2012.08478. [CrossRef]
42. Yang, S.; Tu, K. Bottom-Up Constituency Parsing and Nested Named Entity Recognition with Pointer Networks. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Dublin, Ireland, 22–27 May 2022; Association for Computational Linguistics: Dublin, Ireland, 2022; pp. 2403–2416. [CrossRef]
43. Yan, H.; Gui, T.; Dai, J.; Guo, Q.; Zhang, Z.; Qiu, X. A Unified Generative Framework for Various NER Subtasks. *arXiv* **2021**, arXiv:2106.01223.